

Getränkebestellungen am Rattenfest

Studienarbeit
Herbstsemester 2026

Version: Lokaler Build
Datum: 19.09.2026

Team: Andrin Klarer (andrin.klarer@ost.ch)

Jasmin Fässler (jasmin.faessler@ost.ch)

Betreuer: Frieder Loch (frieder.loch@ost.ch)

Departement Informatik
Ostschweizer Fachhochschule



0 Inhaltsverzeichnis

1	Einleitung	3
1.1	Problemstellung	3
1.2	Anwendungsszenarien (detaillierter)	5
1.3	Vorhandene Lösungswege	6
2	Theoretischer Hintergrund	7
2.1	Recherche User Experience während Rattenfest	7
3	Methode	7
3.1	Prototyp / Skizze der Lösung	7
3.2	UI und Backend Konzept (Software-Architektur)	7
4	Ergebnisse	7
5	Diskussion inkl. Limitationen	7
6	Fazit und Ausblick	7
7	Projektinformationen	8
7.1	Projektplan	8
7.2	Langfristiger Zeitplan	8
7.3	Kurzfristiger Plan	9
7.4	Meeting Notizen	9
8	Bilderverzeichnis	10
9	Tabellenverzeichnis	10
10	Bibliografie	10

1 Einleitung

1.1 Problemstellung

Digitalisierung der Getränkeverwaltung während dem Rattenfest zur Effizienzsteigerung während dem Event aber auch bei der Nachbearbeitung (Abrechnung und Buchhaltung). Getränke werden zwischen dem Zentrallager (Kühlwagen) und den Verkaufsstellen (Bars, Backstage) transportiert. Bars (Bar Mitarbeiter) bestellen die Getränke, die Rattenfest Arbeiter stellen die Getränke bereit und die Bar Mitarbeiter holen die Getränke ab.

Vorher mussten Bar Mitarbeiter zum Kühlwagen, ihre Bestellung aufgeben, warten und diese zurücktransportieren. Während dem Rattenfest gibt eine Bar 20-40 Bestellungen auf. Es wurde bereits ein einfacher Prototyp erstellt und getestet. Vor dem Rattenfest erstellen die Bars eine Schätzung an benötigten Getränken, damit das Rattenfest OK offensichtliche Fehler korrigiert und Getränke vorbestellt. Dies wurde bisher mit einem Excel Sheet gelöst.

Ziel: gute Website mit guter Architektur die lange verwendet werden kann und ihren Zweck erfüllt.
Lösung massgeschneidert für das Rattenfest.

1.1.1 These

> wie soll die These aussehen? Wir betrachten sehr viele Dinge in der Arbeit, theoretisch gäbe es mehrere Thesen?

1. Erfolgsszenario vom gesamten Projekt

Die Getränkeverwaltung ist ein Erfolg, wenn es alle Anforderungen sinnvoll erfüllt und die Bedienung für RF Arbeiter und Bar Arbeiter angenehm ist. Es soll möglich sein, die Getränkeverwaltung mit mehr Features zu ergänzen, aber es soll nicht nötig sein, programmierte Teile abzuändern. Das wird erreicht durch gutes Planen und Fokus auf das Wichtigste, wobei eventuell nicht jedes nützliche Feature eingebaut wird.

2. These wie an einer Arbeit?

Es ist möglich ein Getränkeverwaltungssystem für das Rattenfest zu konzeptieren und umzusetzen, welches benutzerfreundlich, erweiterbar und einfach wartbar ist. (sind wahrscheinlich die wichtigsten 3 Anforderungen)

1.1.2 Anforderungen

Es wird eine Lösung gesucht, die einfach nutzbar ist (keine versteckten Buttons, suchen von Funktionen..) und während dem Rattenfest muss es fehlerfrei laufen.

Kompromisse

- Wir setzen den Fokus beim Hosting auf Einfachheit und möglichst wenig Setupaufwand für den Server. Dabei nehmen wir in Kauf, uns von einem Hosting Provider mit fertigem Setup abhängig zu machen.
- Bedienbarkeit VS Performance
 - optimieren von Visualisierungen > Getränke Bilder im Cache halten, CDN, Webstandards nutzen (WebP, SVG), Testen wie gross der Einfluss auf die Performance ist oder eine Option zum deaktivieren der Bilder?

weitere Anforderungen (noch priorisieren) :

- Bedienbarkeit:
 - Schritte (sinnvoll) rückgängig machen können
 - Visualisierungen nutzen
 - weniger Klicks und scrolls oder User nicht überfordern? -> Bar User definieren, z.B. wenn es pro Bar eine verantwortliche Person gibt darf die Bedienung schwieriger sein, damit Zeit gespart werden kann

- auf verschiedenen Geräten und Bildschirmgrößen!!
- Umstellung auf Digital verbessern
 - Pausenfunktion einbinden für RF Arbeiter, da jetzt Meldungen sofort reinkommen
 - den Bars das einschätzen der Bestellmenge vereinfachen
 - neben der individuellen Auswahl (+1, +6, +24 ...) auch vordefinierte Mengen als Option anbieten. Dazu auch die Überlegung: eine Bestellung von vorher wiederholen zu können.
 - (Bilder nutzen: bei grosser Bestellung ein Bild mit vielen Getränken und bei wenigen Getränken Bild mit wenig Getränken (z.B. mit ein paar vordefinierten Bildern und die einordnen, müsste recherchieren wie das andere lösen))
 - (Bestellung per Touchscreen, andere Bedienfunktionen machen am Rattenfest keinen Sinn. Die Altersgruppe kennt sich mit dem Smartphone aus.)
- Weitere mögliche Bedienbarkeit Features für Bars
 - Getränke als Favoriten vermerken (automatisch die vorbestellten Getränke markieren) ohne das dieses Feature im Weg ist
 - nach Alkohol und Alkoholfrei filtern (wäre das nützlich?) und sortieren nach Kriterien
 - OK Notfallmeldung per Chat/Telefon/Treffen ermöglichen für Ausnahmefälle und Probleme > einfach halten
 - wenn Website nicht läuft, eine Telefonnummer einblenden die auch offline angezeigt wird.
- wartbarkeit:
 - OK Mitglieder sollten das System einfach updaten können (aber evt. Anleitung erstellen anstatt alles automatisieren) > einfacher geworden mit KI
 - Software best practices anwenden (z.B. Tests)
 - das Meiste soll in einem GUI gelöst werden können, DB Anpassungen nicht per SQL/Table Editor nötig.
- erweiterbarkeit: weitere ineffiziente Prozessabläufe vom Rattenfest digitalisieren/automatisieren ist nicht wichtig, soll möglich sein aber kann auch separat umgesetzt werden. Kommunikation im OK, Website für das Rattenfest, Ticketsystem und weiteres bleibt getrennt.
 - Code Komplexität oder Erweiterbarkeit wichtiger? Zukunftspläne?
- langfristige Lösung: natürlich sollen neue Getränke erfasst werden können, aber auch jährliche Zahlen speichern für Analysen in Zukunft und erweiterbarkeit durch weitere Tools?
- Performance:
 - sollte nicht zu teuer sein, wenn es nur am Rattenfest intensiv genutzt wird, bei ein paar Sekunden längerer Verarbeitungszeit nicht schlimm?
 - das initiale UI und Klicks sollten sofort erkannt werden und sinnvolles Feedback gezeigt werden (lade-animation, Erfolgsmeldung, Überlegung PWA)
 - Internetverbindung während Rattenfest wichtig
 - nicht so langsam, wie ein Webshop sich leisten kann
 - Kurze Teilnehmerumfrage zeigt, dass das Internet während dem Rattenfest an der OST langsamer ist als sonst
- Verbindung
 - Die Nutzer sollten die Verbindung zur Application beibehalten können während dem Rattenfest (Websockets, SSE genügt nicht(?)), so dass sie Status-Nachrichten sofort erhalten ohne regelmässiges abfragen im Hintergrund. Nachrichten sollen nicht verloren gehen aber eine zu aufwändige Überprüfung (mehrfaches hin-und-zurück zum ob Nachricht ankam) soll nicht die Performance durch erhöhten Traffic verschlechtern (Mittelmass finden).
 - Trotz frequenten Bestellungen, soll der Lagerbestand immer aktuell sein. Verzögerung bei Bars sind weniger schlimm, als Verzögerungen beim Lagersystem.
 - Datenverkehr: App zu Lager und Bar = Anzahl Getränke vorhanden App zu Bar = Bestellung Ok/ Nicht Ok, Bestellung bereit zum abholen App zu Lager = Bestellung erhalten

1. allfällige Undo/Stornieren Commands

- Push Nachrichten auf dem Smartphone anzeigen mit PWA wäre ein nützliches Feature.
- Fehler einfach beheben:
 - Hinweise anzeigen, bei ungewöhnlichen Bestellungen (eine 0 zuviel bei Anzahl Getränke, Feld leer gelassen welches bei diesem Getränk sonst eingetragen wird, besonders kleine Bestellung..)
 - Rückgängig/Wiederholen Button genügend gross und schnell (gutes System für History finden)
- Zuverlässigkeit : sehr wichtig, keine halb verarbeiteten Bestellungen, keine falschen Informationen anzeigen
- Redundanz und Skalierbarkeit ist weniger wichtig, da die Anzahl User sich im < 100 Personen Bereich befinden.

1.1.3 Functional Requirements

1.1.3.1 RF Admin

- Admin Dashboard

Immer:

- Analyse/Statistics: was wäre wichtig? Export als Excel/CSV der Daten (wie? wo speichern?)
Vergleiche zwischen verschiedenen Jahren. Bessere Schätzungen
- Getränke im Lager erfassen/anpassen
 - sinnvolle Optionen/Felder
- Bar User erstellen und teilen

während Fest:

- Bestellung erhalten, als bereit markieren, stornieren (mit Bemerkung? mit Flags (nichtabholung/Falschbestellung?)),

1.1.3.2 Bar

- Login mit erhaltenen Daten
- Getränke bestellen (Bar)
- Undo/Redo (siehe oben)
- History > alte Bestellungen betrachten, prüfen, wiederholen
- Abholung > Identität verifizieren nötig? Abholung bestätigen
- Kontaktfunktion (allgemeine Probleme/Fragen melden, Notfallmeldungen während Fest, wer von RF behandelt Meldungen?)

1.1.4 Risiken und Unsicherheiten

Risiko, dass es auf einer Plattform/Betriebssystem/Bildschirmgröße nicht funktioniert wie gedacht

Risiko, dass es nicht alle Personen einrichten können

Spezielle Umgebung: während Rattenfest

- ausserhalb von der Website: Tipps für Bars, wie sie das Gerät mit der Website einrichten können, damit es möglichst integriert und nicht lästig ist (Gerät, Halterung, Rolle zuweisen ?)
- Bedienung von Website (User Experience) eng, dunkel, laut, Bedienung eingeschränkt

1.2 Anwendungsszenarien (detaillierter)

> in welcher Form?

1. Vorbereitung & Inventar

- Ersteinlagerung: Die gesamte eingekaufte Ware wird digital erfasst und bildet das zentrale Live-Inventar.

2. Bestellprozess für Bars

- Zugang: Jede Bar erhält einen eigenen Zugangslink (z.B. app.com/UUID).
- Bestelllogik:
 - Bars bestellen flexibel nach Bedarf.
 - Nachbestellungen über die ursprüngliche Schätzung hinaus sind möglich, solange der Artikel im zentralen Inventar verfügbar ist.
 - Optional: Ein automatisches Warnsystem meldet den Organisatoren kritisch tiefe Lagerbestände, erlaubt nur noch eine bestimmte Anzahl pro Bestellung (Falls eigenes Limit bereits erreicht ist).

3. Abwicklung & Abholung

1. Bestellung: Die Bar sendet eine Bestellung über die App ab.
 - Bestellung für wenige verschiedene Getränke
 - Bestellung für ein Getränk in grossen Mengen
 - eine Bestellung von vorher anpassen und nochmals bestellen
 - a. Bar kann die Bestellung stornieren, anpassen und neu senden.
2. Benachrichtigung: Das RF-Team wird sofort digital informiert.
3. Bereitstellung: Das Team stellt die Ware zusammen und markiert die Bestellung in der App als «Abholbereit». a. RF-Team sieht Problem und kann Bestellung stornieren/markieren.
4. Abholung: Die Bar sieht das Status-Update in Echtzeit und holt die Ware ab, was unnötige Wege vermeidet. a. RF-Team kann Bestellung stornieren/markieren wenn zu lange nicht abgeholt.
0. Bar/RF kann bei technischen Problemen eine Person vom RF mit einer Notfallmeldung erreichen.

4. Prozess für Getränke Rücknahmen

1. Bars können wie bei einer Bestellung eine Rückgabe erfassen und bringen die Ware zum Lager.
2. Admin-Funktion: Nur ein Admin kann Rücknahmen bearbeiten... (Wichtig bei Rückgabe von Schrumpfpackungen, da diese nur als Einheit angenommen werden)
5. Erweitert Analytics, Umsatz pro Bar, pro Getränk etc. Export von Bezugslisten, Bestellhistorie, Rechnung etc. Initiale Bestellung, Getränkewünsche ebenfalls über die App > Analyse, ob ein Getränk viel zu wenig gekauft wurde und durch ein trendigeres Getränk ersetzen vor Schritt 1?

Weitere Features, z.B. «3.1 Bestellung» «die Bestellung für später vorbereiten und noch nicht senden, RF kann diese bereits sehen und sich besser vorbereiten»

- Konkurrenzanalyse durchführen, schauen was die gut machen und wir übernehmen können, ergänzen.
- User Befragung, wäre es sinnvoll oder unnötig
- Abwägen wie gross der Aufwand für den Nutzen ist

1.3 Vorhandene Lösungswege

> konkrete gut dokumentierte Lösungen finden

- event organisations softwares > meiste haben nicht fokus Getränkbestellung sondern tickets, zeitmanagement..
- Getränkelieferanten/Webshop Lösungen (Ziel: wenig Performance, es läuft das ganze Jahr durch)
- Lösungen ohne Website?
- der Vorteil gegenüber einem Chat wo man Bestellungen schreibt ist offensichtlich, aber freie Textnachrichten haben auch Vorteile > Bemerkungen-Möglichkeit bei Bestellung einbauen?

1.3.1 Webshops

<https://www.rewe.de/shop/c/getraenke-genussmittel/>

- Angenehme Navigation durch farbige Bilder und Icons
 - einzelne Getränke und 6er Packs werden jeweils so auf dem Bild abgebildet

2 Theoretischer Hintergrund

> Unterschied zur Einleitung?? ist das der Kontext?

2.1 Recherche User Experience während Rattenfest

2.1.1 Definition

2.1.2 Erkenntnisse

2.1.3 wichtige Probleme

2.1.4 Konkrete Massnahmen

> oder gehört das zu Methode/Konzept?

2.1.4.1 Design

2.1.4.2 Software-Architektur

3 Methode

3.1 Prototyp / Skizze der Lösung

3.2 UI und Backend Konzept (Software-Architektur)

4 Ergebnisse

5 Diskussion inkl. Limitationen

6 Fazit und Ausblick

7 Projektinformationen

7.1 Projektplan

Der Aufwand beträgt 2 Arbeitstage pro Woche für ca. 17h/W , insgesamt 240 Stunden Aufwand.

7.2 Langfristiger Zeitplan

7.2.0.1 M1: Recherche fertig

geschätzte Zeit: xxh benötigte Zeit: xxh

Beschreibung: {Link zu Kapitel in Typst?}

- Recherche erstellt, damit die Anforderungen bestmöglichst priorisiert
- User Tests durchgeführt mit einem Prototyp??
 - neues UI/UX Design oder vorhandenen Prototyp nutzen?
- User Befragung mit Bar-Arbeitern und RF Lager Verwaltern
- Konzept inklusive Application Architektur und Software Architektur anhand der Anforderungen erstellt

7.2.0.2 M2: Umsetzung, Programmierung abgeschlossen

Beschreibung:

- FR umgesetzt
- NFR umgesetzt/erreicht
- Tests durchgeführt und Rückmeldungen eingebaut
- Website erreichbar und benutzbar
- allfällige Dokumentation/Anleitungen für User (RF/Bars) fertig

7.2.0.3 M3: Dokumentation bereit zum Korrekturlesen

- Erkenntnisse dokumentiert
- Dokumentation gut formatiert
- Abgabe bereit
- Dokumentation kann korrekturgelesen werden
- 1. Puffer

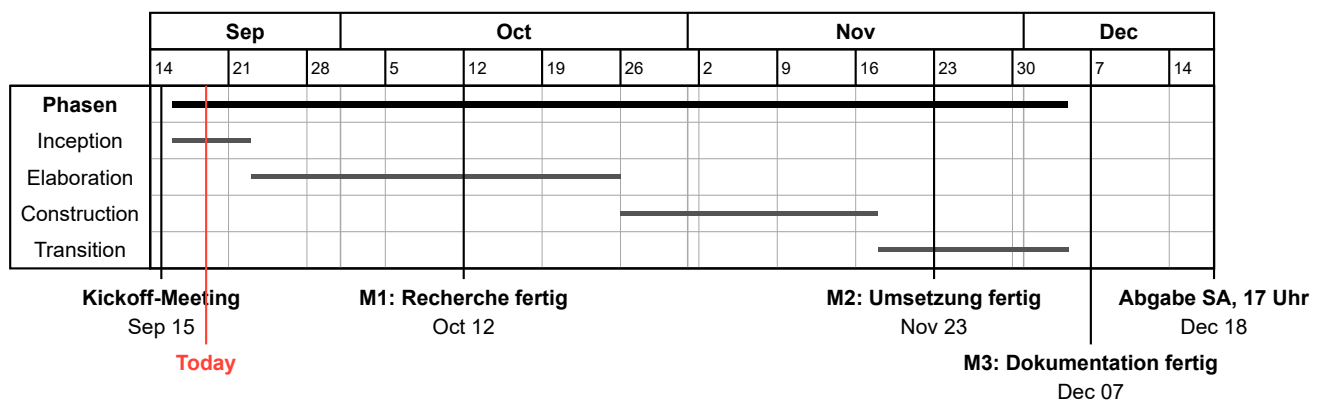


Abbildung 1: Langfristiger Zeitplan

7.3 Kurzfristiger Plan

Punkte die erledigt werden pro Woche, Struktur und ausführlichkeit kann sich ändern. Als Hilfe für uns.

Woche 1:

- Kickoff
- Dokumentationsstruktur aufbauen + Link zu PDF
- Aufgabe konkretisieren/Arbeit grob planen
- Erfolgsszenario definieren
- Zeitplan
- offene Fragen notieren für nächste Besprechung: sind mit > am Anfang geschrieben.

Woche 2:

-

7.3.1 Kanban Board

Backlog (2)	In Arbeit (0)	In Review / Done (2)
Dokumentationsstruktur anpassen/ ergänzen + Requirements genauer definieren doc A		Typst aufsetzen, Dokumentationsstruktur erstellen und erstes Brainstorming doc J
Anforderungen priorisieren: wie? und was weglassen?, These verbessern > benötige Feedback ?, Struktur verbessern > benötige Feedback, ob richtig verstanden, Anwendungsszenarien für User anpassen? und weitere definieren mit Prioritätsstatus doc J		Anpassungen von Andrin übernehmen, erste Erfolgsszenario/ Anwendungsszenarien doc J

7.4 Meeting Notizen

Meetingnotizen werden separat erstellt, hier werden nur wichtige Entscheidungen notiert.

- Wöchentliches Meeting am Dienstag mit Betreuer

8 Bilderverzeichnis

Abbildung 1 Langfristiger Zeitplan	8
--	---

9 Tabellenverzeichnis

10 Bibliografie